

-- =====

-- HOMEWORK: CLASS 5 - CTEs, PIVOT, EXPRESSIONS & WINDOW FUNCTIONS (EASY VERSION)

-- Database: BikeStores Sample Database

-- Instructions: Write SQL statements to solve each problem below.

-- ===== ASSIGNMENT_5 =====

Name : NAHEED DEEDAR

ID : 801825

-- =====

-- SECTION A: CASE Expressions

-- =====

-- Q1: Write a simple CASE that shows order_status as a word instead of number.

-- Show order_id, order_status (number), and status_description (word).

SQLQuery3.sql...ujitsu (53))* SQLQuery2.sql...fujitsu (56))*

```
1  SELECT
2      order_id,
3      order_status,
4      CASE order_status
5          WHEN 1 THEN 'Pending'
6          WHEN 2 THEN 'Processing'
7          WHEN 3 THEN 'Rejected'
8          WHEN 4 THEN 'Completed'
9          ELSE 'Unknown'
10     END AS status_description
11 FROM sales.orders;
```

100 % No issues found

Results Messages

	order_id	order_status	status_description
1	1	4	Completed
2	2	4	Completed
3	3	4	Completed
4	4	4	Completed
5	5	4	Completed
6	6	4	Completed
7	7	4	Completed
8	8	4	Completed
9	9	4	Completed
10	10	4	Completed
11	11	4	Completed

Query executed successfully. DESKTOP-26F

-- Q2: Categorize products by price:

-- Under \$500 = 'Budget'

-- \$500 to \$2000 = 'Standard'

-- Over \$2000 = 'Premium'

-- Show product_name, list_price, and price_category.

1-SQLQueryA...ujitsu (53))*
SQLQuery2.sql...fujitsu (56))*

```

1  SELECT
2      product_name,
3      list_price,
4      CASE
5          WHEN list_price < 500 THEN 'Budget'
6          WHEN list_price BETWEEN 500 AND 2000 THEN 'Standard'
7          WHEN list_price > 2000 THEN 'Premium'
8          ELSE 'Unknown'
9      END AS price_category
10 FROM production.products;

```

100 %
No issues found

Results
Messages

	product_name	list_price	price_category
1	Trek 820 - 2016	379.99	Budget
2	Ritchey Timberwolf Frameset - 2016	749.99	Standard
3	Surly Wednesday Frameset - 2016	999.99	Standard
4	Trek Fuel EX 8 29 - 2016	2899.99	Premium
5	Heller Shagamaw Frame - 2016	1320.99	Standard
6	Surly Ice Cream Truck Frameset - 2016	469.99	Budget
7	Trek Slash 8 27.5 - 2016	3999.99	Premium
8	Trek Remedy 29 Carbon Frameset - 2016	1799.99	Standard
9	Trek Conduit+ - 2016	2999.99	Premium
10	Surly Straggler - 2016	1549.00	Standard
11	Surly Straggler 650b - 2016	1680.99	Standard

Query executed successfully.
DESKTOP-26P7AE7 (17.0 RTM)

- Q3: Using CASE with COUNT, count how many orders have status = 4 (Completed)
- vs non-completed for each store. Show store_id, completed_count, not_completed_count.

1-SQLQueryA...ujitsu (52))* SQLQuery2.sql...fujitsu (51))*

```
3  -- vs non-completed for each store. Show store_id, completed_count, not_co
4  use BikeStores;
5
6  SELECT
7      store_id,
8
9      COUNT(CASE WHEN order_status = 4 THEN 1 END) AS completed_count,
10
11     COUNT(CASE WHEN order_status <> 4 THEN 1 END) AS non_completed_count
12
13 FROM sales.orders
14 GROUP BY store_id;
```

100 % No issues found

Results Messages

	store_id	completed_count	non_completed_count
1	3	142	32
2	1	284	64
3	2	1019	74

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTOP-26P7AE7\fujits... BikeStore

-- Q4: Create a column called "year_label" that shows:

-- If model_year = 2024: 'New'

-- If model_year = 2023: 'Recent'

-- Else: 'Older'

-- Show product_name, model_year, year_label.

1-SQLQueryA...ujitsu (52))*
SQLQuery2.sql...fujitsu (51))*

```

7
8 SELECT
9     product_name,
10    model_year,
11
12 CASE
13     WHEN model_year = 2024 THEN 'New'
14     WHEN model_year = 2023 THEN 'Recent'
15     Else 'Older'
16 END AS year_label
17
18 FROM
19     [production].[products]

```

100 %
No issues found

Results
Messages

	product_name	model_year	year_label
1	Trek 820 - 2016	2016	Older
2	Ritchey Timberwolf Frameset - 2016	2016	Older
3	Surly Wednesday Frameset - 2016	2016	Older
4	Trek Fuel EX 8 29 - 2016	2016	Older
5	Heller Shagamaw Frame - 2016	2016	Older
6	Surly Ice Cream Truck Frameset - 2016	2016	Older
7	Trek Slash 8 27.5 - 2016	2016	Older
8	Trek Remedy 29 Carbon Frameset - 2016	2016	Older
9	Trek Conduit+ - 2016	2016	Older
10	Surly Straggler - 2016	2016	Older
11	Surly Straggler 650b - 2016	2016	Older

Query executed successfully.
DESKTOP-26P7A

-- Q5: For customers, show email and a column called "has_email" that says 'Yes' if email is not NULL, 'No' if NULL.

4-SQLQueryA...ujitsu (52))*
SQLQuery2.sql...fujitsu (51))*

```

1
2  -- Q5: For customers, show email and a column called "has_email"
3  -- that says 'Yes' if email is not NULL, 'No' if NULL.
4  SELECT
5      customer_id,
6      CONCAT(first_name, ' ', last_name) AS full_name,
7      CASE
8          WHEN email IS NULL THEN 'No'
9          ELSE 'Yes'
10     END AS has_email
11  FROM sales.customers;

```

100 %
No issues found

Results
Messages

	customer_id	full_name	has_email
1	1	Debra Burks	Yes
2	2	Kasha Todd	Yes
3	3	Tameka Fisher	Yes
4	4	Daryl Spence	Yes
5	5	Charolette Rice	Yes
6	6	Lyndsey Bean	Yes
7	7	Latasha Hays	Yes
8	8	Jacqueline Duncan	Yes
9	9	Genoveva Baldwin	Yes
10	10	Pamelia Newman	Yes
11	11	Deshawn Mendoza	Yes

Query executed successfully.
DESKTOP-26P7AE7 (17.0 RTM) | DESKTOP-26P7AE7\ufujits...

-- =====

-- SECTION B: CTEs (Common Table Expressions)

-- =====

-- Q6: Create a CTE called "high_value_products" that selects products with list_price > 3000.

-- Then SELECT from that CTE to show all those products.

5-SQLQueryA...ujitsu (54))*
SQLQuery2.s...ot connected*

```

3      --      Then SELECT from that CTE to show all those products.
4      --use BikeStores;
5      WITH high_value_products AS (
6          SELECT
7              product_id,
8              product_name,
9              model_year,
10             list_price
11         FROM production.products
12         WHERE list_price > 3000
13     )
14     SELECT *
15     FROM high_value_products;
16
17
18

```

100 %
No issues found

Results
Messages

	product_id	product_name	model_year	list_price
1	7	Trek Slash 8 27.5 - 2016	2016	3999.99
2	40	Trek Fuel EX 9.8 29 - 2017	2017	4999.99
3	43	Trek Fuel EX 9.8 27.5 Plus - 2017	2017	5299.99
4	47	Trek Remedy 9.8 - 2017	2017	5299.99
5	49	Trek Domane SL 6 - 2017	2017	3499.99
6	50	Trek Silque SLR 7 Women's - 2017	2017	5999.99
7	51	Trek Silque SLR 8 Women's - 2017	2017	6499.99
8	54	Trek Domane SL Disc Frameset - 2017	2017	3199.99
9	56	Trek Domane SLR 6 Disc - 2017	2017	5499.99

Query executed successfully.
DESKTOP-26P7AE7 (17.0 RTM)
DESKTOP-26P7AE7\fu

-- Q7: Write a CTE that calculates the average list_price of all products.

-- Then use it to find products that cost more than average.

First Query:

The screenshot shows a SQL Server Enterprise Manager interface. At the top, there are three tabs: 'SQLQuery4.sql...fujitsu (52)*', '6-SQLQueryA...ujitsu (54)*', and 'SQLQuery2.s...ot connected*'. The active window displays a SQL query with line numbers 1 through 17. The query is a CTE that calculates the average list price of all products and then selects products with a list price greater than that average. Below the query editor, there is a status bar showing '100 %' and 'No issues found'. At the bottom, there is a 'Results' tab and a 'Messages' tab. The 'Results' tab is active, showing a table with 4 columns: 'product_id', 'product_name', and 'list_price'. The table contains 7 rows of data. At the very bottom, a yellow status bar indicates 'Query executed successfully.' and shows the server name 'DESKTOP-26P7AE7 (17.0 RTM)' and the user 'BikeStores'.

```
1
2  -- Q7: Write a CTE that calculates the average list_price of all products.
3  --      Then use it to find products that cost more than average.
4
5  WITH avg_price AS (
6      SELECT AVG(list_price) AS average_price
7      FROM production.products
8  )
9  SELECT
10     product_id,
11     product_name,
12     list_price
13  FROM production.products
14  WHERE list_price > (
15      SELECT average_price
16      FROM avg_price
17  );
```

	product_id	product_name	list_price
1	4	Trek Fuel EX 8 29 - 2016	2899.99
2	7	Trek Slash 8 27.5 - 2016	3999.99
3	8	Trek Remedy 29 Carbon Frameset - 2016	1799.99
4	9	Trek Conduit+ - 2016	2999.99
5	10	Surly Straggler - 2016	1549.00
6	11	Surly Straggler 650b - 2016	1680.99
7	28	Surly Karate Monkey 27.5+ Frameset - 2017	2499.99

Query executed successfully. | DESKTOP-26P7AE7 (17.0 RTM) | DESKTOP-26P7AE7\fujitsu... | BikeStores

2nd Query:

7-SQLQueryAs...ujitsu (54) SQLQuery2.s...ot connected*

```

1
2 -- Q7: Write a CTE that calculates the average list_price of all products.
3 --     Then use it to find products that cost more than average.
4
5 WITH avg_price AS (
6     SELECT AVG(list_price) AS average_price
7     FROM production.products
8 )
9 SELECT
10     p.product_id,
11     p.product_name,
12     p.list_price,
13     a.average_price
14 FROM production.products p
15 CROSS JOIN avg_price a
16 WHERE p.list_price > a.average_price;

```

100 % No issues found

Results Messages

	product_id	product_name	list_price	average_price
1	4	Trek Fuel EX 8 29 - 2016	2899.99	1520.591401
2	7	Trek Slash 8 27.5 - 2016	3999.99	1520.591401
3	8	Trek Remedy 29 Carbon Frameset - 2016	1799.99	1520.591401
4	9	Trek Conduit+ - 2016	2999.99	1520.591401
5	10	Surly Straggler - 2016	1549.00	1520.591401
6	11	Surly Straggler 650b - 2016	1680.99	1520.591401
7	28	Surly Karate Monkey 27.5+ Frameset - 2017	2499.99	1520.591401

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTOP-26P7AE7\fujitsu... BikeStores

- Q8: Create a CTE called "customer_order_counts" that counts how many orders each customer has.
- Then use it to find customers with more than 5 orders.

```
SQLQuery5.sql...fujitsu (54))* 7-SQLQueryA...ujitsu (57))* SQLQuery2.s...ot connected*
1
2 -- Q8: Create a CTE called "customer_order_counts" that counts how many orders each customer has.
3 -- Then use it to find customers with more than 5 orders.
4
5 WITH customer_order_counts AS (
6     SELECT
7         customer_id,
8         COUNT(*) AS order_count
9     FROM sales.orders
10    GROUP BY customer_id
11 )
12 SELECT
13     customer_id,
14     order_count
15 FROM customer_order_counts
16 WHERE order_count > 5;
17
18
100 % No issues found Ln: 16, Ch: 23 SPC CRLF
Results Messages
customer_id order_count
Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTOP-26P7AE7\fujits... BikeStores 00:00:00 Row: 1, Col: 1 0 ro
```

-- =====

-- SECTION C: ROW_NUMBER() and RANK() - EASY BEGINNER

-- =====

-- Q9: Use ROW_NUMBER() to number all products ordered by list_price from highest to lowest.

-- Show product_name, list_price, and row_number.

SQLQuery5.s...ot connected* 7-SQLQueryA...ujitsu (78))* SQLQuery2.s...ot connected*

```
1
2
3 -- Q9: Use ROW_NUMBER() to number all products ordered by list_price from highest to lowest.
4 --     Show product_name, list_price, and row_number.
5
6
7 SELECT
8     product_name,
9     list_price,
10    ROW_NUMBER() OVER (ORDER BY list_price DESC) AS row_number
11 FROM production.products;
```

100 % No issues found Ln: 11, Ch: 26 SPC CRL

Results Messages

	product_name	list_price	row_number
1	Trek Domane SLR 9 Disc - 2018	11999.99	1
2	Trek Domane SLR 8 Disc - 2018	7499.99	2
3	Trek Domane SL Frameset - 2018	6499.99	3
4	Trek Domane SL Frameset Women's - 2018	6499.99	4
5	Trek Emonda SLR 8 - 2018	6499.99	5
6	Trek Silque SLR 8 Women's - 2017	6499.99	6
7	Trek Silque SLR 7 Women's - 2017	5999.99	7

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTOP-26P7AE7\fujitsu... BikeStores 00:00:00 Row: 1, Col: 1 32

-- Q10: Use ROW_NUMBER() to rank products by price WITHIN each brand (partition by brand_id).

-- Show brand_id, product_name, list_price, and rank_in_brand.

SQLQuery5.s...ot connected* 8-SQLQueryA...ujitsu (78))* SQLQuery2.s...ot connected*

```

1
2  -- Q10: Use ROW_NUMBER() to rank products by price WITHIN each brand (partition by brand_id).
3  -- Show brand_id, product_name, list_price, and rank_in_brand.
4  SELECT
5      brand_id,
6      product_name,
7      list_price,
8      ROW_NUMBER() OVER (
9          PARTITION BY brand_id
10         ORDER BY list_price DESC
11     ) AS rank_in_brand
12 FROM production.products;

```

100 % No issues found Ln: 12, Ch: 26 SPC CRLF

Results Messages

	brand_id	product_name	list_price	rank_in_brand
1	1	Electra Townie Commute Go! - 2018	2999.99	1
2	1	Electra Townie Commute Go! Ladies' - 2018	2999.99	2
3	1	Electra Townie Commute Go! - 2018	2999.99	3
4	1	Electra Townie Commute Go! Ladies' - 2018	2999.99	4
5	1	Electra Loft Go! 8i - 2018	2799.99	5
6	1	Electra Townie Go! 8i - 2017/2018	2599.99	6
7	1	Electra Townie Go! 8i Ladies' - 2018	2599.99	7

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTOP-26P7AE7/fujitsu... BikeStores 00:00:00 Row: 1, Col: 1 321

-- Q11: Use RANK() instead of ROW_NUMBER() on products ordered by list_price.

-- See what happens when multiple products have the same price.

SQLQuery5.s...ot connected* 10-SQLQuery...ujitsu (78))* SQLQuery2.s...ot connected*

```

1
2  -- Q11: Use RANK() instead of ROW_NUMBER() on products ordered by list_price.
3  --      See what happens when multiple products have the same price.
4
5  SELECT
6      brand_id,
7      product_name,
8      list_price,
9      RANK() OVER (
10         ORDER BY list_price DESC
11     ) AS rank_in_price
12 FROM production.products;

```

100 % No issues found Ln

Results Messages

	brand_id	product_name	list_price	rank_in_price
1	9	Trek Domane SLR 9 Disc - 2018	11999.99	1
2	9	Trek Domane SLR 8 Disc - 2018	7499.99	2
3	9	Trek Domane SL Frameset - 2018	6499.99	3
4	9	Trek Domane SL Frameset Women's - 2018	6499.99	3
5	9	Trek Emonda SLR 8 - 2018	6499.99	3
6	9	Trek Silque SLR 8 Women's - 2017	6499.99	3
7	9	Trek Silque SLR 7 Women's - 2017	5999.99	7

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTOP-26P7AE7\fujitsu... BikeStores 00:00:

```

-- =====
-- SECTION D: Window Functions - Running Totals and Averages
-- =====

```

```

-- Q12: Calculate a running total of daily orders (cumulative sum over time).
--      Show order_date, daily_order_count, and running_total.

```

SQLQuery5.s...ot connected*

11-SQLQueryA...ujitsu (52)

SQLQuery2.s...ot connected*

```
1
2  -- Q12: Calculate a running total of daily orders (cumulative sum over time).
3  --      Show order_date, daily_order_count, and running_total.
4
5  SELECT
6      order_date,
7      Change - Not Saved ) AS daily_order_count,
8      SUM(COUNT(*)) OVER (
9          ORDER BY order_date
10     ) AS running_total
11  FROM sales.orders
12  GROUP BY order_date;
```

100 %

No issues found



Results



Messages

	order_date	daily_order_count	running_total
1	2016-01-01	2	2
2	2016-01-02	1	3
3	2016-01-03	2	5
4	2016-01-04	3	8
5	2016-01-05	3	11
6	2016-01-06	1	12
7	2016-01-08	1	13

Query executed successfully.

DESKTOP-26P7AE7 (17.0 RTM)

DESKTOP-26P7AE7\fujitsu...

BikeStores | 00

SQLQuery5.s...ot connected* 11-SQLQueryA...ujitsu (52) SQLQuery2.s...ot connected*

```

1
2 -- Q12: Calculate a running total of daily orders (cumulative sum over time).
3 -- Show order_date, daily_order_count, and running_total.
4
5 WITH daily_orders AS (
6     SELECT
7         order_date,
8         COUNT(*) AS daily_order_count
9     FROM sales.orders
10    GROUP BY order_date
11 )
12 SELECT
13     order_date,
14     daily_order_count,
15     SUM(daily_order_count) OVER (ORDER BY order_date) AS running_total
16 FROM daily_orders;

```

100 % No issues found Ln: 4, Ch:

Results Messages

	order_date	daily_order_count	running_total
1	2016-01-01	2	2
2	2016-01-02	1	3
3	2016-01-03	2	5
4	2016-01-04	3	8
5	2016-01-05	3	11
6	2016-01-06	1	12
7	2016-01-08	1	13

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTOP-26P7AE7\fujitsu... BikeStores 00:00:00 Row

-- Q13: For each product, show its list_price and the average list_price of its brand.

-- Use AVG() OVER (PARTITION BY brand_id).

SQLQuery5.s...ot connected* 11-SQLQuery...ujitsu (52))* SQLQuery2.s...ot connected*

```

1
2  -- Q13: For each product, show its list_price and the average list_price of its brand.
3  --      Use AVG() OVER (PARTITION BY brand_id).
4  SELECT
5      product_name,
6      brand_id,
7      list_price,
8      AVG(list_price) OVER (
9          PARTITION BY brand_id
10         ) AS avg_brand_price
11  FROM production.products;

```

100 % No issues found Ln: 8, Ch: 27 S

Results Messages

	product_name	brand_id	list_price	avg_brand_price
1	Electra Townie Original 21D - 2016	1	549.99	761.006186
2	Electra Cruiser 1 (24-Inch) - 2016	1	269.99	761.006186
3	Electra Girl's Hawaii 1 (16-inch) - 2015/2016	1	269.99	761.006186
4	Electra Moto 1 - 2016	1	529.99	761.006186
5	Electra Townie Original 7D EQ - 2016	1	599.99	761.006186
6	Electra Townie Original 7D EQ - Women's - 2016	1	599.99	761.006186
7	Electra Cruiser 1 (24-Inch) - 2016	1	269.99	761.006186

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTOP-26P7AE7\fujitsu... BikeStores 00:00:00 Row: 1, C

-- Q14: Calculate a running total of quantity sold for each product over time.

-- Show product_id, order_date, quantity, and cumulative_quantity for that product.

SQLQuery5.s...ot connected* 12-SQLQuery...ujitsu (52))* SQLQuery2.s...ot connected*

```

1
2 -- Q14: Calculate a running total of quantity sold for each product over time.
3 -- Show product_id, order_date, quantity, and cumulative_quantity for that product.
4 SELECT
5     oi.product_id,
6     o.order_date,
7     oi.quantity,
8     SUM(oi.quantity) OVER (
9         PARTITION BY oi.product_id
10        ORDER BY o.order_date
11    ) AS cumulative_quantity
12 FROM sales.order_items oi
13 JOIN sales.orders o
14     ON oi.order_id = o.order_id;

```

100 % No issues found Ln: 14, Ch: 33 SPC

Results Messages

	product_id	order_date	quantity	cumulative_quantity
1	2	2016-01-03	2	2
2	2	2016-01-14	2	4
3	2	2016-01-18	1	5
4	2	2016-02-05	1	6
5	2	2016-02-09	1	7
6	2	2016-02-26	1	8
7	2	2016-02-28	1	10

-- =====

-- SECTION E: LAG, LEAD (Previous and Next)

-- =====

-- Q15: For each customer, show their order date and the date of their previous order.

-- Use LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date).

SQLQuery5.s...ot connected* 13-SQLQuery...ujitsu (52))* SQLQuery2.s...ot connected*

```

1
2 -- Q15: For each customer, show their order date and the date of their previous order.
3 -- Use LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date).
4 SELECT
5     customer_id,
6     order_date,
7     LAG(order_date) OVER (
8         PARTITION BY customer_id
9         ORDER BY order_date
10    ) AS previous_order_date
11 FROM sales.orders;

```

100 % No issues found Ln: 11, Ch: 19 SPC

Results Messages

	customer_id	order_date	previous_order_date
1	1	2016-12-09	NULL
2	1	2018-04-18	2016-12-09
3	1	2018-11-18	2018-04-18
4	2	2017-02-05	NULL
5	2	2017-08-21	2017-02-05
6	2	2018-04-09	2017-08-21
7	3	2018-03-27	NULL

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTOP-26P7AE7\fujitsu... BikeStores 00:00:00 Row: 1, Col: 1

-- Q16: Calculate the number of days between a customer's consecutive orders.

-- (Use LAG and DATEDIFF)

SQLQuery5.s...ot connected* 15-SQLQueryA...ujitsu (52) SQLQuery2.s...ot connected*

```

2  -- Q16: Calculate the number of days between a customer's consecutive orders.
3  -- (Use LAG and DATEDIFF)
4  SELECT
5      customer_id,
6      order_date,
7      LAG(order_date) OVER (
8          PARTITION BY customer_id
9          ORDER BY order_date
10     ) AS previous_order_date,
11     DATEDIFF(
12         day,
13         LAG(order_date) OVER (
14             PARTITION BY customer_id
15             ORDER BY order_date
16         ),
17         order_date
18     ) AS days_between_orders
19 FROM sales.orders;

```

100 % No issues found Ln: 19, Ch: 1

Results Messages

	customer_id	order_date	previous_order_date	days_between_orders
1	1	2016-12-09	NULL	NULL
2	1	2018-04-18	2016-12-09	495
3	1	2018-11-18	2018-04-18	214
4	2	2017-02-05	NULL	NULL
5	2	2017-08-21	2017-02-05	197
6	2	2018-04-09	2017-08-21	231

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTOP-26P7AE7\fujitsu... BikeStores 00:00:00 Row:

-- =====

-- SECTION F: PIVOT (Rows to Columns)

-- =====

-- Q17: Create a simple pivot showing the count of orders for each order_status (1,2,3,4)

-- as separate columns. Only need store_id and the 4 status columns.

SQLQuery5.s...ot connected* 16-SQLQueryA...ujitsu (52)) SQLQuery2.s...ot connected*

```

4
5 SELECT
6     store_id,
7     [1] AS status_1,
8     [2] AS status_2,
9     [3] AS status_3,
10    [4] AS status_4
11 FROM
12 (
13     SELECT
14         store_id,
15         order_status
16     FROM sales.orders
17 ) AS src
18 PIVOT
19 (
20     COUNT(order_status)
21     FOR order_status IN ([1], [2], [3], [4])
22 ) AS p;

```

100 % No issues found

Results Messages

	store_id	status_1	status_2	status_3	status_4
1	3	8	5	19	142
2	1	25	27	12	284
3	2	29	31	14	1019

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTOP-26P7AE7\fujitsu... BikeSt

-- =====

-- SECTION G: Mixed Practice (Putting It All Together)

-- =====

-- Q18: Use CASE to categorize customers by total spending:

-- Over \$5000 = 'VIP'

-- \$1000-\$5000 = 'Regular'

-- Under \$1000 = 'New'

-- Show customer_name and tier.

SQLQuery5.s...ot connected* 17-SQLQueryA...ujitsu (52) SQLQuery2.s...ot connected*

```

1  -- Q18: Use CASE to categorize customers by total spending:
2  --     Over $5000 = 'VIP'
3  --     $1000-$5000 = 'Regular'
4  --     Under $1000 = 'New'
5  --     Show customer_name and tier.
6
7  SELECT
8      c.first_name + ' ' + c.last_name AS customer_name,
9      CASE
10         WHEN SUM(oi.quantity * oi.list_price * (1 - oi.discount)) > 5000
11             THEN 'VIP'
12         WHEN SUM(oi.quantity * oi.list_price * (1 - oi.discount)) >= 1000
13             THEN 'Regular'
14         ELSE 'New'
15     END AS tier
16 FROM sales.customers c
17 JOIN sales.orders o
18     ON c.customer_id = o.customer_id
19 JOIN sales.order_items oi
20     ON o.order_id = oi.order_id
21 GROUP BY
22     c.customer_id,
23     c.first_name,
24     c.last_name;

```

100 % No issues found Ln: 9, Cl

Results Messages

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTOP-26P7AE7\ujitsu... BikeStores 00:00:00 Row

	customer_name	tier
1	Debra Burks	VIP
2	Kasha Todd	VIP
3	Tameka Fisher	VIP
4	Daryl Spence	VIP
5	Charolette Rice	VIP
6	Lyndsey Bean	VIP
7	Latasha Hays	VIP
8	Jacqueline Dun...	R...
9	Genoveva Bal...	VIP
10	Pamelia New...	VIP
11	Deshawn Me...	R...
12	Robby Sykes	VIP
13	Lashawn Ortiz	VIP

Query executed successfully.

-- Q19: Use ROW_NUMBER() and CASE together: Find top 3 products per category,

-- and label them as 'Gold', 'Silver', 'Bronze'.

```
SQLQuery5.s...ot connected* 18-SQLQueryA...ujitsu (52) SQLQuery2.s...ot connected*
2  -- and label them as 'Gold', 'Silver', 'Bronze'.
3
4  WITH ranked_products AS
5  (
6      SELECT
7          category_id,
8          product_name,
9          list_price,
10         ROW_NUMBER() OVER
11         (
12             PARTITION BY category_id
13             ORDER BY list_price DESC
14         ) AS rn
15         FROM production.products
16     )
17     SELECT
18         category_id,
19         product_name,
20         list_price,
21         CASE rn
22             WHEN 1 THEN 'Gold'
23             WHEN 2 THEN 'Silver'
24             WHEN 3 THEN 'Bronze'
25         END AS medal
26     FROM ranked_products
27     WHERE rn <= 3;
```

100 % No issues found

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTOP-26P7AE7\fujitsu... BikeStores 0C

27 WHERE rn <= 3;

100 % No issues found

	category_id	product_name	list_price	medal
1	1	Electra Straight 8 3i (20-inch) - Boy's - 2017	489.99	Gold
2	1	Electra Townie 3i EQ (20-inch) - Boys' - 2017	489.99	Silver
3	1	Trek Superfly 24 - 2017/2018	489.99	Bronze
4	2	Electra Townie Go! 8i - 2017/2018	2599.99	Gold
5	2	Electra Townie Commute 27D - 2018	899.99	Silver
6	2	Electra Townie Commute 27D Ladies - 2018	899.99	Bronze
7	3	Electra Townie Commute Go! - 2018	2999.99	Gold
8	3	Electra Townie Commute Go! Ladies' - 2018	2999.99	Silver
9	3	Electra Townie Go! 8i Ladies' - 2018	2599.99	Bronze
10	4	Trek Boone 7 Disc - 2018	3999.99	Gold

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM)

-- Q20: Create a CTE that calculates monthly revenue, then use LAG to show month-over-month growth.

SQLQuery5.s...ot connected*

19-SQLQueryA...ujitsu (52))

SQLQuery2.s...ot connected*

```
1
2 -- Q20: Create a CTE that calculates monthly revenue,
3 -- then use LAG to show month-over-month growth.
4 WITH monthly_revenue AS
5 (
6     SELECT
7         YEAR(o.order_date) AS order_year,
8         MONTH(o.order_date) AS order_month,
9         SUM(
10             oi.quantity *
11             oi.list_price *
12             (1 - oi.discount)
13         ) AS revenue
14     FROM sales.orders o
15     JOIN sales.order_items oi
16     ON o.order_id = oi.order_id
17     GROUP BY
18         YEAR(o.order_date),
19         MONTH(o.order_date)
20 )
21 SELECT
22     order_year,
23     order_month,
24     revenue,
25     LAG(revenue) OVER
26     (
27         ORDER BY order_year, order_month
28     ) AS previous_month_revenue
```

100 % No issues found

Query executed successfully.

DESKTOP-26P7AE7 (17.0 RTM) | DESKTOP-26P7AE7\fujitsu... | BikeSto

```

SQLQuery5.s...ot connected* 19-SQLQueryA...ujitsu (52) SQLQuery2.s...ot connected*
9
10         oi.quantity *
11         oi.list_price *
12         (1 - oi.discount)
13     ) AS revenue
14 FROM sales.orders o
15 JOIN sales.order_items oi
16     ON o.order_id = oi.order_id
17 GROUP BY
18     YEAR(o.order_date),
19     MONTH(o.order_date)
20 )
21 SELECT
22     order_year,
23     order_month,
24     revenue,
25     LAG(revenue) OVER
26     (
27         ORDER BY order_year, order_month
28     ) AS previous_month_revenue,
29
30     revenue -
31     LAG(revenue) OVER
32     (
33         ORDER BY order_year, order_month
34     ) AS revenue_growth
35
36 FROM monthly_revenue;

```

100 % No issues found Ln: 3

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) | DESKTOP-26P7AE7\fujitsu... | BikeStores | 00:00:

27 || ORDER BY order_year, order_month

100 % No issues found

Results Messages

	order_year	order_month	revenue	previous_month_revenue	revenue_growth
1	2016	1	215146.4241	NULL	NULL
2	2016	2	156112.3228	215146.4241	-59034.1013
3	2016	3	180600.3285	156112.3228	24488.0057
4	2016	4	167144.0512	180600.3285	-13456.2773
5	2016	5	205270.0091	167144.0512	38125.9579
6	2016	6	210562.1245	205270.0091	5292.1154
7	2016	7	199556.8089	210562.1245	-11005.3156
8	2016	8	225657.3767	199556.8089	26100.5678
9	2016	9	273091.6097	225657.3767	47434.2330
10	2016	10	212078.0805	273091.6097	-61013.5292
11	2016	11	182329.4124	212078.0805	-29748.6681
12	2016	12	199829.9792	182329.4124	17500.5668
13	2017	1	285616.4840	199829.9792	85786.5048
14	2017	2	312923.7470	285616.4840	27307.2630
15	2017	3	308911.9018	312923.7470	-4011.8452

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) | DESKTOP-26P7AE7\fi

- Q21: Write a query that shows each product, its price, its rank in its brand,
- and a CASE that says 'Top Product' if rank = 1, else 'Other'.

SQLQuery5.s...ot connected*

20-SQLQuery...ujitsu (52))*

SQLQuery2.s...ot connected*

```

1
2  -- Q21: Write a query that shows each product, its price, its rank in its brand,
3  --       and a CASE that says 'Top Product' if rank = 1, else 'Other'.
4  WITH ranked_products AS
5  (
6      SELECT
7          product_name,
8          brand_id,
9          list_price,
10         ROW_NUMBER() OVER
11         (
12             PARTITION BY brand_id
13             ORDER BY list_price DESC
14         ) AS rank_in_brand
15     FROM production.products
16 )
17 SELECT
18     product_name,
19     list_price,
20     rank_in_brand,
21     CASE
22         WHEN rank_in_brand = 1 THEN 'Top Product'
23         ELSE 'Other'
24     END AS product_status
25 FROM ranked_products;

```

100 %

No issues found

Ln: 25, Ch: 22

Query executed successfully.

DESKTOP-26P7AE7 (17.0 RTM) | DESKTOP-26P7AE7\fujitsu... | BikeStores | 00:00:00 | Row: 1,

Results

Messages

	product_name	list_price	rank_in_brand	product_status
1	Electra Townie Commute Go! - 2018	2999.99	1	Top Product
2	Electra Townie Commute Go! Ladies' - 2018	2999.99	2	Other
3	Electra Townie Commute Go! - 2018	2999.99	3	Other
4	Electra Townie Commute Go! Ladies' - 2018	2999.99	4	Other
5	Electra Loft Go! 8i - 2018	2799.99	5	Other
6	Electra Townie Go! 8i - 2017/2018	2599.99	6	Other
7	Electra Townie Go! 8i Ladies' - 2018	2599.99	7	Other
8	Electra Townie Go! 8i Ladies' - 2018	2599.99	8	Other
9	Electra Townie Go! 8i - 2017/2018	2599.99	9	Other
10	Electra Townie Go! 8i - 2017/2018	2599.99	10	Other
11	Electra Amsterdam Royal 8i - 2017/2018	1259.90	11	Other
12	Electra Amsterdam Royal 8i Ladies - 2018	1199.99	12	Other
13	Electra Amsterdam Fashion 7i Ladies' - 2017	1099.99	13	Other
14	Electra Delivery 3i - 2016/2017/2018	959.99	14	Other
15	Electra Straight 8 3i - 2018	909.99	15	Other

Query executed successfully.

DESKTOP-26P7AE7 (17.0 RTM) | DESK

- Q22: Create a pivot showing the count of customers by state and by customer tier
- (you'll need to create the tier using CASE first, then pivot).

```
1
2 -- Q22: Create a pivot showing the count of customers by state and by customer tier
3 -- (you'll need to create the tier using CASE first, then pivot).
4 WITH customer_spending AS
5 (
6     SELECT
7         c.customer_id,
8         c.state,
9         SUM(oi.quantity * oi.list_price * (1 - oi.discount)) AS total_spending
10    FROM sales.customers c
11    JOIN sales.orders o
12        ON c.customer_id = o.customer_id
13    JOIN sales.order_items oi
14        ON o.order_id = oi.order_id
15    GROUP BY
16        c.customer_id,
17        c.state
18 ),
19 customer_tiers AS
20 (
21     SELECT
22         customer_id,
23         state,
24         CASE
25             WHEN total_spending > 5000 THEN 'VIP'
26             WHEN total_spending >= 1000 THEN 'Regular'
27             ELSE 'New'
28         END AS tier
```

100 %

No issues found

Ln: 42, Ch: 16

Query executed successfully.

DESKTOP-26P7AE7 (17.0 RTM)

DESKTOP-26P7AE7\fujitsu...

BikeStores

00:00:00 Row

```

16         c.customer_id,
17         c.state
18     ),
19     customer_tiers AS
20     (
21         SELECT
22             customer_id,
23             state,
24             CASE
25                 WHEN total_spending > 5000 THEN 'VIP'
26                 WHEN total_spending >= 1000 THEN 'Regular'
27                 ELSE 'New'
28             END AS tier
29         FROM customer_spending
30     )
31     SELECT
32         state,
33         [VIP],
34         [Regular],
35         [New]
36     FROM customer_tiers
37     PIVOT
38     (
39         COUNT(customer_id)
40         FOR tier IN ([VIP], [Regular], [New])
41     ) AS p
42     ORDER BY state;

```

100 % No issues found

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTOP-26P7AE7/fujitsu... BikeStores 0

100 % No issues found

Results Messages

	product_name	list_price	rank_in_brand	product_status
1	Electra Townie Commute Go! - 2018	2999.99	1	Top Product
2	Electra Townie Commute Go! Ladies' - 2018	2999.99	2	Other
3	Electra Townie Commute Go! - 2018	2999.99	3	Other
4	Electra Townie Commute Go! Ladies' - 2018	2999.99	4	Other
5	Electra Loft Go! 8i - 2018	2799.99	5	Other
6	Electra Townie Go! 8i - 2017/2018	2599.99	6	Other
7	Electra Townie Go! 8i Ladies' - 2018	2599.99	7	Other
8	Electra Townie Go! 8i Ladies' - 2018	2599.99	8	Other
9	Electra Townie Go! 8i - 2017/2018	2599.99	9	Other
10	Electra Townie Go! 8i - 2017/2018	2599.99	10	Other
11	Electra Amsterdam Royal 8i - 2017/2018	1259.90	11	Other
12	Electra Amsterdam Royal 8i Ladies - 2018	1199.99	12	Other
13	Electra Amsterdam Fashion 7i Ladies' - 2017	1099.99	13	Other

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTOP-26P7

-- =====

-- END OF HOMEWORK - ALL QUESTIONS ARE BEGINNER-FRIENDLY

-- =====